



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch11 SQL概述

Software Institute
Nanjing University
Bei Jia

SQL

- SQL (Structured Query Language) 结构化查询语言，是关系数据库的标准语言
- SQL是一个通用的、功能极强的关系数据库语言
 - 综合统一
 - 高度非过程化
 - 面向集合的操作方式
 - 以同一种语法结构提供两种使用方法
 - 语言简洁，易学易用

SQL的特点 1

- 综合统一

- 集数据定义语言（DDL），数据操纵语言（DML），数据控制语言（DCL）功能于一体。
- 可以独立完成数据库生命周期中的全部活动：
 - ◆ 定义和修改、删除关系模式，定义和删除视图，插入数据，建立数据库；
 - ◆ 对数据库中的数据进行查询和更新；
 - ◆ 数据库重构和维护
 - ◆ 数据库安全性、完整性控制，以及事务控制
 - ◆ 嵌入式SQL和动态SQL定义
- 用户数据库投入运行后，可根据需要随时逐步修改模式，不影响数据库的运行。
- 数据操作符统一

SQL的特点 2

- 高度非过程化

- 非关系数据模型的数据操纵语言“面向过程”，必须指定存取路径。
- SQL只要提出“做什么”，无须了解存取路径。
- 存取路径的选择以及SQL的操作过程由系统自动完成。

- 面向集合的操作方式

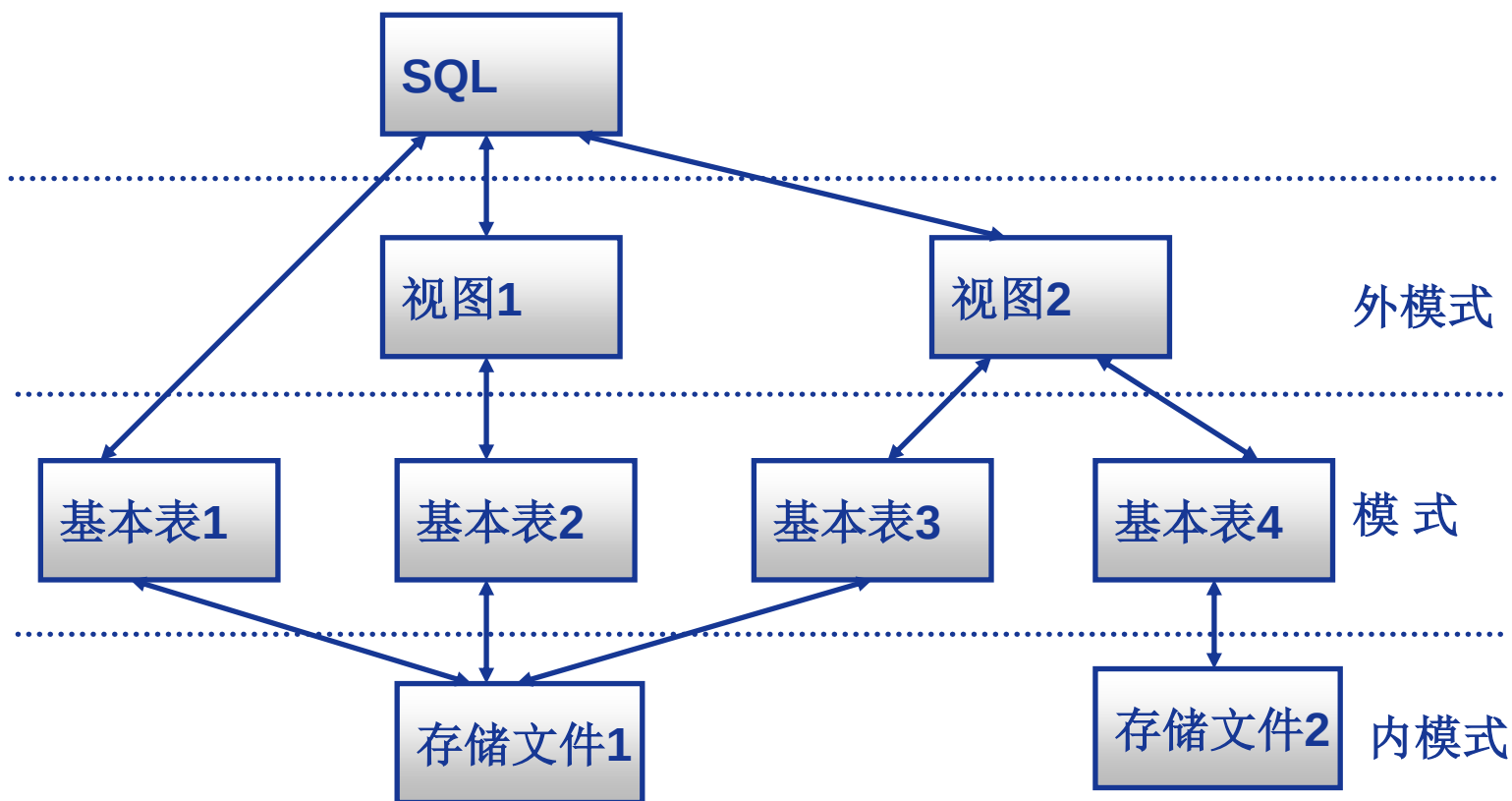
- 非关系数据模型采用面向记录的操作方式，操作对象是一条记录
- SQL采用集合操作方式
 - ◆ 操作对象、查找结果可以是元组的集合
 - ◆ 一次插入、删除、更新操作的对象可以是元组的集合

SQL的特点 3

- 以同一种语法结构提供多种使用方式
 - SQL是独立的语言，能够独立地用于联机交互的使用方式
 - SQL又是嵌入式语言，能够嵌入到高级语言（例如C，C++，Java）程序中，供程序员设计程序时使用
- 语言简洁，易学易用
 - SQL功能极强，完成核心功能只用了9个动词：
 - ◆ 数据定义：CREATE，DROP，ALTER
 - ◆ 数据查询：SELECT
 - ◆ 数据操作：INSERT，UPDATE，DELETE
 - ◆ 数据控制：GRANT，REVOKE



SQL与关系数据库三级模式 1



SQL与关系数据库三级模式 2

- 基本表

- 本身独立存在的表
- 一个关系对应一个基本表
- 一个（或多个）基本表对应一个存储文件
- 一个表可以带若干索引

- 存储文件

- 逻辑结构组成了关系数据库的内模式
- 物理结构对用户是隐蔽的

- 视图

- 从一个或几个基本表导出的表
- 数据库中只存放视图的定义而不存放视图对应的数据
- 视图是一个虚表
- 用户可以在视图上再定义视图



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



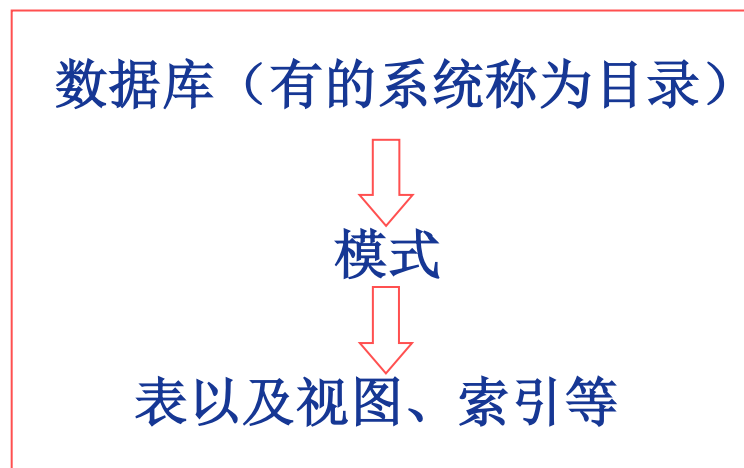
数据管理基础

ch12 SQL数据定义

Software Institute
Nanjing University
Bei Jia

层次化的数据库对象命名机制

- 一个关系数据库管理系统的实例（Instance）中可以建立多个数据库
- 一个数据库中 can 建立多个模式
- 一个模式下通常包括多个表、视图和索引等数据库对象



数据定义

- SQL的数据定义功能:

- 模式定义

- 表定义

- 视图和索引的定义

操 作 对 象	操 作 方 式		
	创 建	删 除	修 改
模式	CREATE SCHEMA	DROP SCHEMA	
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视图	CREATE VIEW	DROP VIEW	
索引	CREATE INDEX	DROP INDEX	ALTER INDEX

定义模式 1

- 定义模式实际上定义了一个命名空间。
 - 在这个空间中可以定义该模式包含的数据库对象，例如基本表、视图、索引等。
- 在CREATE SCHEMA中可以接受CREATE TABLE, CREATE VIEW和GRANT子句。
 - CREATE SCHEMA <模式名>
AUTHORIZATION <用户名> [<表定义子句>|
<视图定义子句>|<授权定义子句>]



定义模式 2

- [例3.1] 为用户WANG定义一个学生-课程模式S-T
 - CREATE SCHEMA “S_T” AUTHORIZATION WANG;
- [例3.2] CREATE SCHEMA AUTHORIZATION WANG;
 - 该语句没有指定<模式名>, <模式名>隐含为<用户名>
- [例3.3]为用户ZHANG创建了一个模式TEST, 并且在其中定义一个表TAB1
 - CREATE SCHEMA TEST AUTHORIZATION ZHANG
CREATE TABLE TAB1 (COL1 SMALLINT,
COL2 INT,
COL3 CHAR(20),
COL4 NUMERIC(10,3),
COL5 DECIMAL(5,2));

删除模式

- DROP SCHEMA <模式名> <CASCADE|RESTRICT>
 - CASCADE（级联）
 - ◆ 删除模式的同时把该模式中所有的数据库对象全部删除
 - RESTRICT（限制）
 - ◆ 如果该模式中定义了下属的数据库对象（如表、视图等），则拒绝该删除语句的执行。
 - ◆ 仅当该模式中没有任何下属的对象时才能执行。
- [例3.4] DROP SCHEMA ZHANG CASCADE;
 - 删除模式ZHANG
 - 同时该模式中定义的表TAB1也被删除



定义基本表

- CREATE TABLE <表名>

(<列名> <数据类型>[<列级完整性约束条件>]

[,<列名> <数据类型>[<列级完整性约束条件>]]

...

[,<表级完整性约束条件>]);

- <表名>：所要定义的基本表的名字
- <列名>：组成该表的各个属性（列）
- <列级完整性约束条件>：涉及相应属性列的完整性约束条件
- <表级完整性约束条件>：涉及一个或多个属性列的完整性约束条件
- 如果完整性约束条件涉及到该表的多个属性列，则必须定义在表级上，否则既可以定义在列级也可以定义在表级。

定义基本表-学生表Student

- [例3.5] 建立“学生”表Student。学号是主码，姓名取值唯一。

```
CREATE TABLE Student
```

```
(Sno CHAR(9) PRIMARY KEY,
```

```
/* 列级完整性约束条件,Sno是主码*/
```

```
Sname CHAR(20) UNIQUE, /* Sname取唯一值*/
```

```
Ssex CHAR(2),
```

```
Sage SMALLINT,
```

```
Sdept CHAR(20)
```

```
);
```

主码

UNIQUE
约束

定义基本表-课程表Course

- [例3.6] 建立一个“课程”表Course

```
CREATE TABLE Course
```

```
(Cno    CHAR(4) PRIMARY KEY,
```

```
  Cname CHAR(40) NOT NULL,
```

```
  Cpno   CHAR(4),
```

先修课

```
  Ccredit SMALLINT,
```

```
  FOREIGN KEY (Cpno) REFERENCES Course(Cno)
```

```
);
```

Cpno是外码
被参照表是**Course**
被参照列是**Cno**



定义基本表-学生选课表SC

- [例3.7] 建立一个学生选课表SC

```
CREATE TABLE SC
```

```
(Sno CHAR(9),
```

```
Cno CHAR(4),
```

```
Grade SMALLINT,
```

```
PRIMARY KEY (Sno,Cno),
```

```
/* 主码由两个属性构成，必须作为表级完整性进行定义*/
```

```
FOREIGN KEY (Sno) REFERENCES Student(Sno),
```

```
/* 表级完整性约束条件，Sno是外码，被参照表是Student */
```

```
FOREIGN KEY (Cno)REFERENCES Course(Cno)
```

```
/* 表级完整性约束条件，Cno是外码，被参照表是Course*/
```

```
);
```



数据类型 1

- SQL中域的概念用数据类型来实现
- 定义表的属性时需要指明其数据类型及长度
- 选用哪种数据类型
 - 取值范围
 - 要做哪些运算



数据类型 2

数据类型	含义
CHAR(<i>n</i>), CHARACTER(<i>n</i>)	长度为 <i>n</i> 的定长字符串
VARCHAR(<i>n</i>), CHARACTERVARYING(<i>n</i>)	最大长度为 <i>n</i> 的变长字符串
CLOB	字符串大对象
BLOB	二进制大对象
INT, INTEGER	长整数（4字节）
SMALLINT	短整数（2字节）
BIGINT	大整数（8字节）
NUMERIC(<i>p</i> , <i>d</i>)	定点数，由 <i>p</i> 位数字（不包括符号、小数点）组成，小数后面有 <i>d</i> 位数字
DECIMAL(<i>p</i> , <i>d</i>), DEC(<i>p</i> , <i>d</i>)	同NUMERIC
REAL	取决于机器精度的单精度浮点数
DOUBLE PRECISION	取决于机器精度的双精度浮点数
FLOAT(<i>n</i>)	可选精度的浮点数，精度至少为 <i>n</i> 位数字
BOOLEAN	逻辑布尔量
DATE	日期，包含年、月、日，格式为YYYY-MM-DD
TIME	时间，包含一日的时、分、秒，格式为HH:MM:SS
TIMESTAMP	时间戳类型
INTERVAL	时间间隔类型



修改基本表 1

- ALTER TABLE <表名>

[ADD[COLUMN] <新列名> <数据类型> [完整性约束]]

[ADD <表级完整性约束>]

[DROP [COLUMN] <列名> [CASCADE | RESTRICT]]

[DROP CONSTRAINT<完整性约束名>[RESTRICT | CASCADE]]

[ALTER COLUMN <列名><数据类型>] ;



修改基本表 2

- <表名>是要修改的基本表
- **ADD**子句用于增加新列、新的列级完整性约束条件和新的表级完整性约束条件
- **DROP COLUMN**子句用于删除表中的列
 - 如果指定了CASCADE短语, 则自动删除引用了该列的其他对象
 - 如果指定了RESTRICT短语, 则如果该列被其他对象引用, 关系数据库管理系统将拒绝删除该列
- **DROP CONSTRAINT**子句用于删除指定的完整性约束条件
- **ALTER COLUMN**子句用于修改原有的列定义, 包括修改列名和数据类型



修改基本表 3

- [例3.8] 向Student表增加“入学时间”列，其数据类型为日期型
 - ALTER TABLE Student ADD S_entrance DATE;
 - 不管基本表中原来是否已有数据，新增加的列一律为空值
- [例3.9] 将年龄的数据类型由字符型（假设原来的数据类型是字符型）改为整数。
 - ALTER TABLE Student ALTER COLUMN Sage INT;
- [例3.10] 增加课程名称必须取唯一值的约束条件。
 - ALTER TABLE Course ADD UNIQUE(Cname);



删除基本表 1

- DROP TABLE <表名> [RESTRICT|CASCADE] ;

- RESTRICT: 删除表是有限制的。

- ◆ 欲删除的基本表不能被其他表的约束所引用

- ◆ 如果存在依赖该表的对象，则此表不能被删除

- CASCADE: 删除该表没有限制。

- ◆ 在删除基本表的同时，相关的依赖对象一起删除



删除基本表 2

- [例3.11] 删除Student表

```
DROP TABLE Student CASCADE;
```

- 基本表定义被删除，数据被删除
- 表上建立的索引、视图、触发器等一般也将被删除

- [例3.12] 若表上建有视图，选择RESTRICT时表不能删除;选择CASCADE时可以删除表，视图也自动删除。

- CREATE VIEW IS_Student

```
AS
```

```
SELECT Sno,Sname,Sage
```

```
FROM Student
```

```
WHERE Sdept='IS';
```

- DROP TABLE Student RESTRICT;

```
--ERROR: cannot drop table Student because other objects depend on it
```

- DROP TABLE Student CASCADE;

```
--NOTICE: drop cascades to view IS_Student
```


索引

- 建立索引的目的：加快查询速度
 - 由数据库管理员或表的拥有者建立
 - 由关系数据库管理系统自动完成维护
 - 关系数据库管理系统自动使用合适的索引作为存取路径，用户不必也不能显式地选择索引
- 关系数据库管理系统中常见索引：
 - 顺序文件上的索引
 - B+树索引
 - 散列（hash）索引
 - 位图索引



建立索引 1

- 语句格式

CREATE [UNIQUE] [CLUSTER] INDEX <索引名>

ON <表名>(<列名>[<次序>][,<列名>[<次序>]]...);

- <表名>: 要建索引的基本表的名字
- 索引: 可以建立在该表的一列或多列上, 各列名之间用逗号分隔
- <次序>: 指定索引值的排列次序, 升序: ASC, 降序: DESC。缺省值: ASC
- UNIQUE: 此索引的每一个索引值只对应唯一的数据记录
- CLUSTER: 表示要建立的索引是聚簇索引



建立索引 2

- [例3.13] 为学生-课程数据库中的Student, Course, SC三个表建立索引。Student表按学号升序建唯一索引, Course表按课程号升序建唯一索引, SC表按学号升序和课程号降序建唯一索引
 - CREATE UNIQUE INDEX Stusno ON Student(Sno);
 - CREATE UNIQUE INDEX Coucno ON Course(Cno);
 - CREATE UNIQUE INDEX SCno ON SC(Sno ASC,Cno DESC);



修改/删除索引

- **ALTER** INDEX <旧索引名> RENAME TO <新索引名>

- [例3.14] 将SC表的SCno索引名改为SCSno

```
ALTER INDEX SCno RENAME TO SCSno;
```

- **DROP** INDEX <索引名>;

- 删除索引时，系统会从数据字典中删去有关该索引的描述。

- [例3.15] 删除Student表的Stusname索引

```
DROP INDEX Stusname;
```



数据字典

- 数据字典是关系数据库管理系统内部的一组系统表，它记录了数据库中所有定义信息：
 - 关系模式定义
 - 视图定义
 - 索引定义
 - 完整性约束定义
 - 各类用户对数据库的操作权限
 - 统计信息等
- 关系数据库管理系统在执行SQL的数据定义语句时，实际上就是在更新数据字典表中的相应信息。





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch13 SQL数据查询（单表）

Software Institute
Nanjing University
Bei Jia

● 语句格式

SELECT [ALL|DISTINCT] <目标
列表达式>[,<目标列表达式>] ...

FROM <表名或视图名>[,<表名
或视图名>]... | (SELECT 语句)

[AS]<别名>

[**WHERE** <条件表达式>]

[**GROUP BY** <列名1> [**HAVING**
<条件表达式>]]

[**ORDER BY** <列名2>
[ASC|DESC]];

- **SELECT**子句：指定要显示的属性列
- **FROM**子句：指定查询对象（基本表或视图）
- **WHERE**子句：指定查询条件
- **GROUP BY**子句：对查询结果按指定列的值分组，该属性列值相等的元组为一个组。通常会在每组中作用聚集函数。
- **HAVING**短语：只有满足指定条件的组才予以输出
- **ORDER BY**子句：对查询结果表按指定列值的升序或降序排序

选择表中的若干列

● 查询指定列

- [例3.16] 查询全体学生的学号与姓名。

```
SELECT Sno,Sname  
FROM Student;
```

- [例3.17] 查询全体学生的姓名、学号、所在系。

```
SELECT Sname,Sno,Sdept  
FROM Student;
```

● 查询全部列

- [例3.18] 查询全体学生的详细记录

```
SELECT  
    Sno,Sname,Ssex,Sage,  
    Sdept  
FROM Student;
```

◆ 或

```
SELECT *  
FROM Student;
```



查询经过计算的值 1

- 查询经过计算的值

- SELECT子句的<目标列表表达式>不仅可以为表中的属性列，也可以是表达式

- [例3.19] 查全体学生的姓名及其出生年份。

```
SELECT Sname,2014-Sage      /*假设当时为2014年*/
```

```
FROM Student;
```

- 输出结果：

Sname	2014-Sage
李勇	1994
刘晨	1995
王敏	1996
张立	1995



查询经过计算的值 2

- [例3.20] 查询全体学生的姓名、出生年份和所在的院系，要求用小写字母表示系名。

```
SELECT Sname,'Year of Birth: ',2014-Sage,LOWER(Sdept)
FROM Student;
```

- 输出结果：

Sname	'Year of Birth:'	2014-Sage	LOWER(Sdept)
李勇	Year of Birth:	1994	cs
刘晨	Year of Birth:	1995	cs
王敏	Year of Birth:	1996	ma
张立	Year of Birth:	1995	is



查询经过计算的值 3

- 使用列别名改变查询结果的列标题:

```
SELECT Sname NAME,'Year of Birth:' BIRTH,  
2014-Sage BIRTHDAY,LOWER(Sdept) DEPARTMENT  
FROM Student;
```

- 输出结果:

NAME	'Year of Birth:'	BIRTH BIRTHDAY	DEPARTMENT
李勇	Year of Birth:	1994	cs
刘晨	Year of Birth:	1995	cs
王敏	Year of Birth:	1996	ma
张立	Year of Birth:	1995	is



选择表中的若干元组 1

- [例3.21] 查询选修了课程的学生学号。

```
SELECT Sno FROM SC;
```

- 等价于:

```
SELECT ALL Sno FROM SC;
```

执行上面的SELECT语句后，结果为：

Sno
201215121
201215121
201215121
201215122
201215122



消除取值重复的行（续）

- 指定DISTINCT关键词，去掉表中重复的行
 - 如果没有指定DISTINCT关键词，则缺省为ALL

```
SELECT DISTINCT Sno
```

```
FROM SC;
```

- 执行结果：

Sno
201215121
201215122



查询满足条件的元组

查询条件	谓词
比较	=, >, <, >=, <=, !=, <>, !>, !<; NOT+上述比较运算符
确定范围	BETWEEN AND, NOT BETWEEN AND
确定集合	IN, NOT IN
字符匹配	LIKE, NOT LIKE
空值	IS NULL, IS NOT NULL
多重条件（逻辑运算）	AND, OR, NOT



① 比较大小

- [例3.22] 查询计算机科学系全体学生的名单。

```
SELECT Sname  
FROM Student  
WHERE Sdept='CS';
```

- [例3.23]查询所有年龄在20岁以下的学生姓名及其年龄。

```
SELECT Sname,Sage  
FROM Student  
WHERE Sage < 20;
```

- [例3.24]查询考试成绩有不及格的学生的学号。

```
SELECT DISTINCT Sn  
FROM SC  
WHERE Grade<60;
```



② 确定范围

- 谓词: BETWEEN ... AND ...

NOT BETWEEN ... AND ...

- [例3.25] 查询年龄在20~23岁（包括20岁和23岁）之间的学生的姓名、系别和年龄

```
SELECT Sname, Sdept, Sage
```

```
FROM Student
```

```
WHERE Sage BETWEEN 20 AND 23;
```

- [例3.26] 查询年龄不在20~23岁之间的学生姓名、系别和年龄

```
SELECT Sname, Sdept, Sage
```

```
FROM Student
```

```
WHERE Sage NOT BETWEEN 20 AND 23;
```



③ 确定集合

- 谓词：IN <值表>, NOT IN <值表>
- [例3.27]查询计算机科学系（CS）、数学系（MA）和信息系（IS）学生的姓名和性别。

```
SELECT Sname, Ssex  
FROM Student  
WHERE Sdept IN ('CS','MA','IS');
```

- [例3.28]查询既不是计算机科学系、数学系，也不是信息系的学生的姓名和性别。

```
SELECT Sname, Ssex  
FROM Student  
WHERE Sdept NOT IN ('IS','MA','CS');
```



④ 字符匹配 1

- 谓词: [NOT] LIKE '<匹配串>' [ESCAPE '<换码字符>']
- <匹配串>可以是一个完整的字符串, 也可以含有通配符% (任意长度 (长度可以为0) 的字符串) 和 _ (任意单个字符)
 - 例如: a%b表示以a开头, 以b结尾的任意长度的字符串
 - 例如: a_b表示以a开头, 以b结尾的长度为3的任意字符串
- 匹配串为固定字符串
 - [例3.29] 查询学号为201215121的学生的详细情况。

```
SELECT *  
FROM Student  
WHERE Sno LIKE '201215121';
```

- 等价于:

```
SELECT *  
FROM Student  
WHERE Sno = '201215121';
```



④ 字符匹配 2

- 匹配串为含通配符的字符串
- [例3.30] 查询所有姓刘学生的姓名、学号和性别。

```
SELECT Sname, Sno, Ssex  
FROM Student  
WHERE Sname LIKE '刘%';
```

- [例3.31] 查询姓"欧阳"且全名为三个汉字的学生的姓名。

```
SELECT Sname  
FROM Student  
WHERE Sname LIKE '欧阳__';
```

- [例3.32] 查询名字中第2个字为"阳"字的学生的姓名和学号。

```
SELECT Sname, Sno  
FROM Student  
WHERE Sname LIKE '__阳%';
```

- [例3.33] 查询所有不姓刘的学生姓名、学号和性别。

```
SELECT Sname, Sno, Ssex  
FROM Student  
WHERE Sname NOT LIKE '刘%';
```



④ 字符匹配 3

- 使用换码字符将通配符转义为普通字符
 - ESCAPE '\ ' 表示“ \ ”为换码字符
- [例3.34] 查询DB_Design课程的课程号和学分。

```
SELECT Cno, Ccredit
FROM   Course
WHERE  Cname LIKE 'DB\_Design' ESCAPE '\ ';
```

- [例3.35] 查询以"DB_"开头，且倒数第3个字符为 i 的课程的具体情况。

```
SELECT *
FROM   Course
WHERE  Cname LIKE 'DB\__%i\__' ESCAPE '\ ';
```



⑤ 涉及空值的查询

- 谓词：IS NULL 或 IS NOT NULL
 - “IS” 不能用 “=” 代替
- [例3.36] 某些学生选修课程后没有参加考试，所以有选课记录，但没有考试成绩。查询缺少成绩的学生的学号和相应的课程号。

```
SELECT Sno, Cno
FROM SC
WHERE Grade IS NULL
```

- [例3.37] 查所有有成绩的学生学号和课程号。

```
SELECT Sno, Cno
FROM SC
WHERE Grade IS NOT NULL;
```



⑥多重条件查询

- 逻辑运算符：AND和 OR 来连接多个查询条件

- AND的优先级高于OR
- 可以用括号改变优先级

- [例3.38] 查询计算机系年龄在20岁以下的学生姓名。

```
SELECT Sname
FROM Student
WHERE Sdept= 'CS' AND
Sage<20;
```

- [例3.27] 查询计算机科学系 (CS)、数学系 (MA) 和信息系 (IS) 学生的姓名和性别。

```
SELECT Sname, Ssex
FROM Student
WHERE Sdept IN ('CS ','MA ','IS')
```

- 可改写为：

```
SELECT Sname, Ssex
FROM Student
WHERE Sdept= ' CS' OR Sdept= '
MA' OR Sdept= 'IS ';
```



对查询结果排序

- ORDER BY子句

- 可以按一个或多个属性列排序
- 升序: ASC;降序: DESC;缺省值为升序
- 对于空值, 排序时显示的次序由具体系统实现来决定

- [例3.39]查询选修了3号课程的学生
的学号及其成绩, 查询结果按分数
降序排列。

```
SELECT Sno, Grade
FROM SC
WHERE Cno= '3'
ORDER BY Grade DESC;
```

- [例3.40]查询全体学生情况, 查询结
果按所在系的系号升序排列, 同一
系中的学生按年龄降序排列。

```
SELECT *
FROM Student
ORDER BY Sdept, Sage DESC;
```



聚集函数 1

- 聚集函数：

- 统计元组个数

COUNT(*)

- 统计一列中值的个数

COUNT([DISTINCT|ALL] <列名>)

- 计算一列值的总和（此列必须为数值型）

SUM([DISTINCT|ALL] <列名>)

- 计算一列值的平均值（此列必须为数值型）

AVG([DISTINCT|ALL] <列名>)

- 求一列中的最大值和最小值

MAX([DISTINCT|ALL] <列名>)

MIN([DISTINCT|ALL] <列名>)



聚集函数 2

- [例3.41] 查询学生总人数。

```
SELECT COUNT(*)  
FROM Student;
```

- [例3.42] 查询选修了课程的学生人数。

```
SELECT COUNT(DISTINCT  
Sno)  
FROM SC;
```

- [例3.43] 计算1号课程的学生平均成绩。

```
SELECT AVG(Grade)  
FROM SC  
WHERE Cno= ' 1 ';
```

- [例3.44] 查询选修1号课程的学生最高分数。

```
SELECT MAX(Grade)  
FROM SC  
WHERE Cno='1';
```

- [例3.45] 查询学生201215012选修课程的总学分数。

```
SELECT SUM(Ccredit)  
FROM SC, Course  
WHERE Sno='201215012' AND  
SC.Cno=Course.Cno;
```



对查询结果分组 1

- GROUP BY子句分组：

- 细化聚集函数的作用对象

- ◆ 如果未对查询结果分组，聚集函数将作用于整个查询结果
- ◆ 对查询结果分组后，聚集函数将分别作用于每个组
- ◆ 按指定的一列或多列值分组，值相等的为一组



对查询结果分组 2

- [例3.46] 求各个课程号及相应的选课人数。

```
SELECT Cno, COUNT(Sno)
FROM SC
GROUP BY Cno;
```

- 查询结果可能为:

Cno	COUNT(Sno)
1	22
2	34
3	44
4	33
5	48

- [例3.47] 查询选修了3门以上课程的学生学号。

```
SELECT Sno
FROM SC
GROUP BY Sno
HAVING COUNT(*) >3;
```

- **HAVING**短语与WHERE子句的区别:
 - 作用对象不同
 - WHERE子句作用于基表或视图, 从中选择满足条件的元组
 - HAVING短语作用于组, 从中选择满足条件的组。



对查询结果分组 3

- [例3.48] 查询平均成绩大于等于90分的学生学号和平均成绩
- 因为WHERE子句中是不能用聚集函数作为条件表达式，下面的语句是不对的：

```
SELECT Sno, AVG(Grade)
FROM SC
WHERE AVG(Grade)>=90
GROUP BY Sno;
```

- 正确的查询语句应该是：

```
SELECT Sno, AVG(Grade)
FROM SC
GROUP BY Sno
HAVING AVG(Grade)>=90;
```





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch14 SQL数据查询（连接）

Software Institute
Nanjing University
Bei Jia

连接查询

- 连接查询：同时涉及两个以上的表的查询
- 连接条件或连接谓词：用来连接两个表的条件
- 一般格式：
 - [
 - [
- 连接字段：连接谓词中的列名称
 - 连接条件中的各连接字段类型必须是可比的，但名字不必相同



等值连接 / 自然连接查询

- 等值连接：连接运算符为=
- [例 3.49] 查询每个学生及其选修课程的情况

```
SELECT Student.*, SC.*  
FROM Student, SC  
WHERE Student.Sno = SC.Sno;
```

- [例 3.50] 对[例 3.49]用自然连接完成。

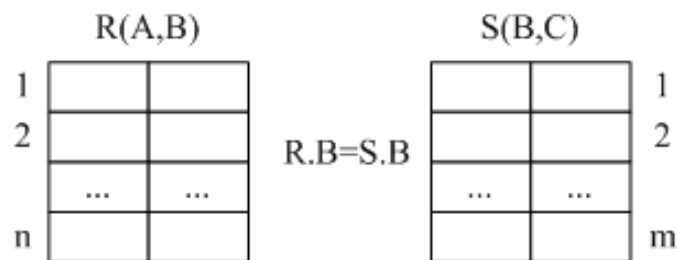
```
SELECT  
Student.Sno, Sname, Ssex, Sage, Sdept,  
Cno, Grade  
FROM Student, SC  
WHERE Student.Sno = SC.Sno;
```

Student.Sno	Sname	Ssex	Sage	Sdept	SC.Sno	Cno	Grade
201215121	李勇	男	20	CS	201215121	1	92
201215121	李勇	男	20	CS	201215121	2	85
201215121	李勇	男	20	CS	201215121	3	88
201215122	刘晨	女	19	CS	201215122	2	90
201215122	刘晨	女	19	CS	201215122	3	80



连接操作的执行过程 1

- 嵌套循环法 (NESTED-LOOP)
 - 首先在表1中找到第一个元组，然后从头开始扫描表2，逐一查找满足连接件的元组，找到后就将表1中的第一个元组与该元组拼接起来，形成结果表中一个元组。
 - 表2全部查找完后，再找表1中第二个元组，然后再从头开始扫描表2，逐一查找满足连接条件的元组，找到后就将表1中的第二个元组与该元组拼接起来，形成结果表中一个元组。
 - 重复上述操作，直到表1中的全部元组都处理完毕



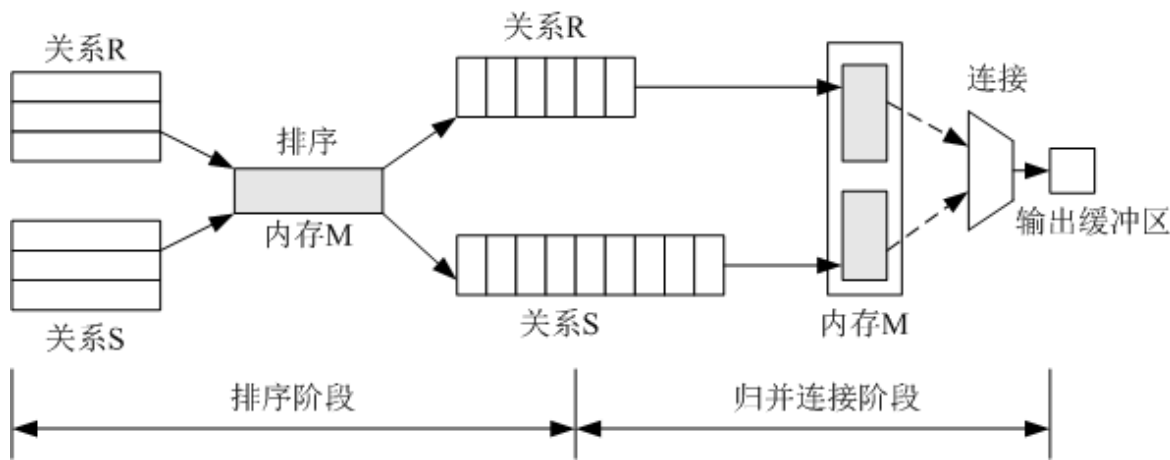
连接操作的执行过程 2

- 排序合并法 (SORT-MERGE)

- 常用于=连接
- 首先按连接属性对表1和表2排序
- 对表1的第一个元组，从头开始扫描表2，顺序查找满足连接条件的元组，找到后就将表1中的第一个元组与该元组拼接起来，形成结果表中一个元组。当遇到表2中第一条大于表1连接字段值的元组时，对表2的查询不再继续

- 索引连接 (INDEX-JOIN)

- 对表2按连接字段建立索引
- 对表1中的每个元组，依次根据其连接字段值查询表2的索引，从中找到满足条件的元组，找到后就将表1中的第一个元组与该元组拼接起来，形成结果表中一个元组



同时进行选择和连接

- 一条SQL语句可以同时完成选择和连接查询，这时WHERE子句是由连接谓词和选择谓词组成的复合条件。
- [例 3.51]查询选修2号课程且成绩在90分以上的所有学生的学号和姓名。

```
SELECT Student.Sno, Sname  
FROM    Student, SC  
WHERE   Student.Sno=SC.Sno AND  
        SC.Cno='2' AND SC.Grade>90;
```

■ 执行过程:

- ◆ 先从SC中挑选出Cno='2'并且Grade>90的元组形成一个中间关系
- ◆ 再和Student中满足连接条件的元组进行连接得到最终的结果关系



自身连接 1

- 自身连接：一个表与其自己进行连接
- 需要给表起别名以示区别
- 由于所有属性名都是同名属性，因此必须使用别名前缀
- [例 3.52]查询每一门课的间接先修课（即先修课的先修课）

```
SELECT FIRST.Cno, SECOND.Cpno  
FROM Course FIRST, Course SECOND  
WHERE FIRST.Cpno = SECOND.Cno;
```



自身连接 2

Course表 (FIRST & SECOND)

课程号	课程名	先行课	学分
Cno	Cname	Cpno	Ccredit
1	课程号	课程名	先行课
2	Cno	Cname	Cpno
3	1	数据库	5
4	2	数学	2
5	3	信息系统	4
6	4	操作系统	3
7	5	数据结构	4
	6	数据处理	2
	7	PASCAL语言	6

查询结果:

Cno	Pcno
1	7
3	5
5	6

外连接 1

● 外连接与普通连接的区别

- 普通连接操作只输出满足连接条件的元组
- 外连接操作以指定表为连接主体，将主体表中不满足连接条件的元组一并输出
 - ◆ 左外连接
 - 列出左边关系中所有的元组
 - ◆ 右外连接
 - 列出右边关系中所有的元组



外连接 2

- [例 3. 53] 改写[例 3.49]

```
SELECT Student.Sno,Sname,Ssex,Sage,Sdept,Cno,Grade
FROM Student LEFT OUT JOIN SC ON
(Student.Sno=SC.Sno);
```

Student.Sno	Sname	Ssex	Sage	Sdept	Cno	Grade
201215121	李勇	男	20	CS	1	92
201215121	李勇	男	20	CS	2	85
201215121	李勇	男	20	CS	3	88
201215122	刘晨	女	19	CS	2	90
201215122	刘晨	女	19	CS	3	80
201215123	王敏	女	18	MA	NULL	NULL
201215125	张立	男	19	IS	NULL	NULL



多表连接

- 多表连接：两个以上的表进行连接
- [例3.54]查询每个学生的学号、姓名、选修的课程名及成绩

```
SELECT Student.Sno, Sname, Cname, Grade  
FROM   Student, SC, Course   /*多表连接*/  
WHERE Student.Sno = SC.Sno  
      AND SC.Cno = Course.Cno;
```





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch15 SQL数据查询（嵌套）

Software Institute
Nanjing University
Bei Jia

嵌套查询概述 1

- 一个SELECT-FROM-WHERE语句称为一个查询块
- 将一个查询块嵌套在另一个查询块的WHERE子句或HAVING短语的条件中的查询称为嵌套查询

```
SELECT Sname                                /*外层查询/父查询*/  
FROM Student  
WHERE Sno IN  
        ( SELECT Sno                        /*内层查询/子查询*/  
          FROM SC  
          WHERE Cno= ' 2 ');
```



嵌套查询概述 2

- 上层的查询块称为外层查询或父查询
- 下层查询块称为内层查询或子查询
- SQL语言允许多层嵌套查询
 - ◆ 即一个子查询中还可以嵌套其他子查询
- 子查询的限制
 - ◆ 不能使用ORDER BY子句
- 有些嵌套查询可以用连接运算替代
 - ◆ 谨慎使用嵌套查询



嵌套查询求解方法

- 不相关子查询：子查询的查询条件不依赖于父查询
 - 由里向外 逐层处理。即每个子查询在上一级查询处理之前求解，子查询的结果用于建立其父查询的查找条件。
- 相关子查询：子查询的查询条件依赖于父查询
 - 首先取外层查询中表的第一个元组，根据它与内层查询相关的属性值处理内层查询，若WHERE子句返回值为真，则取此元组放入结果表
 - 然后再取外层表的下一个元组
 - 重复这一过程，直至外层表全部检查完为止



带有IN谓词的子查询 1

- [例 3.55] 查询与“刘晨”在同一个系学习的学生。

- 此查询要求可以分步来完成

- ① 确定“刘晨”所在系名

```
SELECT Sdept
FROM Student
WHERE Sname= '刘晨';
```

结果为： CS

- ② 查找所有在CS系学习的学生。

```
SELECT Sno, Sname,
Sdept
FROM Student
WHERE Sdept= 'CS';
```

- 结果为：

Sno	Sname	Sdept
201215121	李勇	CS
201215122	刘晨	CS



带有IN谓词的子查询 2

- 将第一步查询嵌入到第二步查询的条件中

```
SELECT Sno, Sname,  
Sdept  
FROM Student  
WHERE Sdept IN  
  (SELECT Sdept  
   FROM Student  
   WHERE Sname= '刘晨');
```

- 此查询为不相关子查询。

- 用自身连接完成[例3.55]查询要求

```
SELECT S1.Sno,  
S1.Sname,S1.Sdept  
FROM Student  
S1,Student S2  
WHERE S1.Sdept =  
S2.Sdept AND  
S2.Sname = '刘晨';
```

带有IN谓词的子查询 3

- [例 3.56]查询选修了课程名为“信息系统”的学生学号和姓名

SELECT Sno,Sname

FROM Student

WHERE Sno IN

(SELECT Sno

FROM SC

WHERE Cno IN

(SELECT Cno

FROM Course

WHERE Cname= '信息系统')

);

③ 最后在Student关系中
取出Sno和Sname

② 然后在SC关系中找到选
修了3号课程的学生学号

① 首先在Course关系中找到
“信息系统”的课程号，为3号



带有IN谓词的子查询 4

- 用连接查询实现[例 3.56]：

```
SELECT Sno,Sname  
FROM   Student,SC,Course  
WHERE  Student.Sno = SC.Sno  AND  
        SC.Cno = Course.Cno AND  
        Course.Cname='信息系统';
```



带有比较运算符的子查询 1

- 当能确切知道内层查询返回单值时，可用比较运算符(>, <, =, >=, <=, !=或< >)。
- 在[例 3.55]中，由于一个学生只可能在一个系学习，则可以用 = 代替IN：

```
SELECT Sno,Sname,Sdept
FROM   Student
WHERE  Sdept =
      (SELECT Sdept
       FROM   Student
       WHERE  Sname= '刘晨');
```



带有比较运算符的子查询 2

- [例 3.57]找出每个学生超过他选修课程平均成绩的课程号。

```
SELECT Sno, Cno
```

```
FROM SC x
```

```
WHERE Grade >=(SELECT AVG (Grade)
```

```
FROM SC y
```

```
WHERE y.Sno=x.Sno);
```

相关子查询



可能的执行过程

- 从外层查询中取出SC的一个元组x，将元组x的Sno值(201215121) 传送给内层查询。

```
SELECT AVG(Grade)
FROM SC y
WHERE y.Sno='201215121';
```

- 执行内层查询，得到值88
- 用该值代替内层查询，得到外层查询：

```
SELECT Sno,Cno
FROM SC x
WHERE Grade >=88;
```

- 执行这个查询，得到
(201215121,1)
(201215121,3)

- 然后外层查询取出下一个元组重复做上述①至③步骤，直到外层的SC元组全部处理完毕。结果为：

(201215121,1)
(201215121,3)
(201215122,2)



带有ANY (SOME) 或ALL谓词的子查询 1

- 使用ANY或ALL谓词时必须同时使用比较运算
- 语义为:
 - $> \text{ANY}$ 大于子查询结果中的某个值
 - $> \text{ALL}$ 大于子查询结果中的所有值
 - $< \text{ANY}$ 小于子查询结果中的某个值
 - $< \text{ALL}$ 小于子查询结果中的所有值
 - $\geq \text{ANY}$ 大于等于子查询结果中的某个值
 - $\geq \text{ALL}$ 大于等于子查询结果中的所有值
 - $\leq \text{ANY}$ 小于等于子查询结果中的某个值
 - $\leq \text{ALL}$ 小于等于子查询结果中的所有值
 - $= \text{ANY}$ 等于子查询结果中的某个值
 - $= \text{ALL}$ 等于子查询结果中的所有值 (通常没有实际意义)
 - $\neq$ (或 $<>$) ANY 不等于子查询结果中的某个值
 - $\neq$ (或 $<>$) ALL 不等于子查询结果中的任何一个值



带有ANY (SOME) 或ALL谓词的子查询 2

- [例 3.58] 查询非计算机科学系中比计算机科学系任意一个学生年龄小的学生姓名和年龄

```
SELECT Sname,Sage
FROM Student
WHERE Sage < ANY (SELECT Sage
                  FROM Student
                  WHERE Sdept= ' CS ')
AND Sdept <> 'CS ' ;
/*父查询块中的条件*/
```

- 结果:

Sname	Sage
王敏	18
张立	19

- 执行过程:

- 首先处理子查询，找出CS系中所有学生的年龄，构成一个集合 (20,19)
- 处理父查询，找所有不是CS系且年龄小于 20 或19的学生



带有ANY (SOME) 或ALL谓词的子查询 3

- 用聚集函数实现[例 3.58]

```
SELECT Sname,Sage
```

```
FROM Student
```

```
WHERE Sage <
```

```
(SELECT MAX (Sage)
```

```
FROM Student
```

```
WHERE Sdept= 'CS ')
```

```
AND Sdept <> ' CS ';
```



带有ANY (SOME) 或ALL谓词的子查询 4

- [例 3.59] 查询非计算机科学系中比计算机科学系所有学生年龄都小的学生姓名及年龄。

- 方法一：用ALL谓词

```
SELECT Sname,Sage
FROM Student
WHERE Sage < ALL
      (SELECT Sage
       FROM Student
       WHERE Sdept= ' CS ')
AND Sdept <> ' CS ';
```

- 方法二：用聚集函数

```
SELECT Sname,Sage
FROM Student
WHERE Sage <
      (SELECT MIN(Sage)
       FROM Student
       WHERE Sdept= ' CS ')
AND Sdept <> ' CS ';
```



带有ANY (SOME) 或ALL谓词的子查询 5

- 表3.7 ANY (或SOME) , ALL谓词与聚集函数、IN谓词的等价转换关系

	=	<>或!=	<	<=	>	>=
ANY	IN	--	<MAX	<=MAX	>MIN	>= MIN
ALL	--	NOT IN	<MIN	<= MIN	>MAX	>= MAX



带有EXISTS谓词的子查询 1

- EXISTS谓词
 - 存在量词 $\exists$
 - 带有EXISTS谓词的子查询不返回任何数据，只产生逻辑真值“true”或逻辑假值“false”。
 - ◆ 若内层查询结果非空，则外层的WHERE子句返回真值
 - ◆ 若内层查询结果为空，则外层的WHERE子句返回假值
 - 由EXISTS引出的子查询，其目标列表表达式通常都用 *，因为带EXISTS的子查询只返回真值或假值，给出列名无实际意义。
- NOT EXISTS谓词
 - 若内层查询结果非空，则外层的WHERE子句返回假值
 - 若内层查询结果为空，则外层的WHERE子句返回真值



带有EXISTS谓词的子查询 2

- [例 3.60]查询所有选修了1号课程的学生姓名。
- 思路分析：
 - 本查询涉及Student和SC关系
 - 在Student中依次取每个元组的Sno值，用此值去检查SC表
 - 若SC中存在这样的元组，其Sno值等于此Student.Sno值，并且其Cno= '1'，则取此Student.Sname送入结果表

SELECT Sname

FROM Student

WHERE EXISTS

(SELECT *

FROM SC

WHERE Sno=Student.Sno AND Cno= ' 1 ');



带有EXISTS谓词的子查询 3

- [例 3.61] 查询没有选修1号课程的学生姓名。

```
SELECT Sname
```

```
FROM   Student
```

```
WHERE NOT EXISTS
```

```
    (SELECT *
```

```
      FROM SC
```

```
      WHERE Sno = Student.Sno AND Cno='1');
```



带有EXISTS谓词的子查询 4

- 不同形式的查询间的替换
 - 一些带EXISTS或NOT EXISTS谓词的子查询不能被其他形式的子查询等价替换
 - 所有带IN谓词、比较运算符、ANY和ALL谓词的子查询都能用带EXISTS谓词的子查询等价替换
- 用EXISTS/NOT EXISTS实现全称量词
 - SQL语言中没有全称量词 $\forall$ (For all)
 - 可以把带有全称量词的谓词转换为等价的带有存在量词的谓词:
$$(\forall x) P \equiv \neg (\exists x (\neg P))$$



带有EXISTS谓词的子查询 5

- [例 3.55]查询与“刘晨”在同一个系学习的学生。

```
SELECT Sno, Sname, Sdept
FROM Student
WHERE Sdept IN
    (SELECT Sdept
     FROM Student
     WHERE Sname= '刘晨');
```

- 可以用带EXISTS谓词的子查询替换:

```
SELECT Sno,Sname,Sdept
FROM Student S1
WHERE EXISTS
    (SELECT *
     FROM Student S2
     WHERE S2.Sdept
     = S1.Sdept AND
     S2.Sname = '刘晨');
```



带有EXISTS谓词的子查询 6

- [例 3.62] 查询选修了全部课程的学生姓名。

■ $(\forall x) P \equiv \neg (\exists x (\neg P))$ 不存在一门课程是该学生不选修的

SELECT Sname

FROM Student

WHERE NOT EXISTS

(SELECT *

FROM Course

WHERE NOT EXISTS

(SELECT * FROM SC

WHERE Sno= Student.Sno

AND Cno= Course.Cno

)

);



带有EXISTS谓词的子查询 7

● [例 3.62] 改

SELECT Sname

FROM student

WHERE Sno IN

(SELECT Sno

FROM SC

Group by Sno

Having count(*) = (select count(*) from course))

带有EXISTS谓词的子查询 7

- 用EXISTS/NOT EXISTS实现逻辑蕴涵
 - SQL语言中没有蕴涵（Implication）逻辑运算
 - 可以利用谓词演算将逻辑蕴涵谓词等价转换为：

$$p \rightarrow q \equiv \neg p \vee q$$



带有EXISTS谓词的子查询 8

- [例 3.63]查询至少选修了学生201215122选修的全部课程的学生号码。
- 解题思路：
 - 用逻辑蕴涵表达：查询学号为x的学生，对所有的课程y，只要201215122学生选修了课程y，则x也选修了y。
 - 形式化表示：
 - ◆ 用p表示谓词“学生201215122选修了课程y”
 - ◆ 用q表示谓词“学生x选修了课程y”
 - ◆ 则上述查询为： $(\forall y) p \rightarrow q$
 - ◆ 等价变换：
$$\begin{aligned}(\forall y) p \rightarrow q &\equiv \neg (\exists y (\neg (p \rightarrow q))) \\ &\equiv \neg (\exists y (\neg (\neg p \vee q))) \\ &\equiv \neg \exists y (p \wedge \neg q)\end{aligned}$$
 - 变换后语义：不存在这样的课程y，学生201215122选修了y，而学生x没有选。

- 用NOT EXISTS谓词表示：

```
SELECT DISTINCT Sno
FROM SC SCX
WHERE NOT EXISTS
    (SELECT *
     FROM SC SCY
     WHERE SCY.Sno = '
201215122' AND
        NOT EXISTS
            (SELECT *
             FROM SC SCZ
             WHERE
SCZ.Sno=SCX.Sno AND
SCZ.Cno=SCY.Cno));
```





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch16 SQL数据查询（集合）

Software Institute
Nanjing University
Bei Jia

集合查询

- 集合操作的种类
 - 并操作UNION
 - 交操作INTERSECT
 - 差操作EXCEPT
- 参加集合操作的各查询结果的列数必须相同；对应项的数据类型也必须相同



并操作

- UNION: 将多个查询结果合并起来时, 系统自动去掉重复元组
- UNION ALL: 将多个查询结果合并起来时, 保留重复元组
- [例 3.64] 查询计算机科学系的学生及年龄不大于19岁的学生。

```
SELECT *  
FROM Student  
WHERE Sdept= 'CS'  
UNION  
SELECT *  
FROM Student  
WHERE Sage<=19;
```
- [例 3.65] 查询选修了课程1或者选修了课程2的学生。

```
SELECT Sno  
FROM SC  
WHERE Cno=' 1 '  
UNION  
SELECT Sno  
FROM SC  
WHERE Cno= ' 2 ';
```



交操作

- [例3.66] 查询计算机科学系的学生与年龄不大于19岁的学生的交集。

```
SELECT *  
FROM Student  
WHERE Sdept='CS'  
INTERSECT  
SELECT *  
FROM Student  
WHERE Sage<=19
```

- [例 3.66] 实际上就是查询计算机科学系中年龄不大于19岁的学生。

```
SELECT *  
FROM Student  
WHERE Sdept= 'CS' AND Sage<=19;
```

- [例 3.67]查询既选修了课程1又选修了课程2的学生。

```
SELECT Sno  
FROM SC  
WHERE Cno=' 1 '  
INTERSECT  
SELECT Sno  
FROM SC  
WHERE Cno='2 ';
```

- [例3.67]也可以表示为:

```
SELECT Sno  
FROM SC  
WHERE Cno=' 1 ' AND Sno IN  
(SELECT Sno  
FROM SC  
WHERE Cno=' 2 ');
```



差操作

- [例 3.68] 查询计算机科学系的学生与年龄不大于19岁的学生的差集。

```
SELECT *  
FROM Student  
WHERE Sdept='CS'  
EXCEPT  
SELECT *  
FROM Student  
WHERE Sage <=19;
```

- [例3.68]实际上是查询计算机科学系中年龄大于19岁的学生

```
SELECT *  
FROM Student  
WHERE Sdept= 'CS' AND  
Sage>19;
```





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch17 SQL数据查询（基于派生表）

Software Institute
Nanjing University
Bei Jia

基于派生表的查询1

- 子查询不仅可以出现在WHERE子句中，还可以出现在FROM子句中，这时子查询生成的临时派生表（Derived Table）成为主查询的查询对象
- [例3.57]找出每个学生超过他自己选修课程平均成绩的课程号

```
SELECT Sno, Cno
FROM SC, (SELECT Sno, Avg(Grade)
          FROM SC
          GROUP BY Sno)
AS Avg_sc(avg_sno,avg_grade)
WHERE SC.Sno = Avg_sc.avg_sno
and SC.Grade >=Avg_sc.avg_grade
```



基于派生表的查询 2

- 如果子查询中没有聚集函数，派生表可以不指定属性列，子查询SELECT子句后面的列名为其缺省属性。
- [例3.60]查询所有选修了1号课程的学生姓名，可以用如下查询完成：

```
SELECT Sname
```

```
FROM Student,
```

```
(SELECT Sno FROM SC WHERE Cno=' 1 ') AS SC1
```

```
WHERE Student.Sno=SC1.Sno;
```





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch18 SQL数据更新-插入

Software Institute
Nanjing University
Bei Jia

插入元组 1

- 将新元组插入指定表中，语句格式

INSERT

INTO <表名> [(<属性列1>[,<属性列2 >...])]

VALUES (<常量1> [,<常量2>]...);

- INTO子句

- 指定要插入数据的表名及属性列
- 属性列的顺序可与表定义中的顺序不一致
- 没有指定属性列：表示要插入的是一条完整的元组，且属性列属性与表定义中的顺序一致
- 指定部分属性列：插入的元组在其余属性列上取空值

- VALUES子句

- 提供的值必须与INTO子句匹配
 - ◆ 值的个数
 - ◆ 值的类型



插入元组 2

- [例3.69]将一个新学生元组（学号：201215128;姓名：陈冬;性别：男;所在系：IS;年龄：18岁）插入到Student表中。

```
INSERT  
INTO Student  
(Sno,Sname,Ssex,Sdept,Sage)  
VALUES ('201215128','陈冬','男','IS',18);
```

- [例3.70]将学生张成民的信息插入到Student表中。

```
INSERT  
INTO Student  
VALUES ('201215126','张成民','男',18,'CS');
```

- [例3.71] 插入一条选课记录（'200215128','1'）。

```
INSERT  
INTO SC(Sno,Cno)  
VALUES ('201215128 ','1 ');
```

- 关系数据库管理系统将在新插入记录的Grade列上自动地赋空值。
- 或者：

```
INSERT  
INTO SC  
VALUES (' 201215128 ','1 ',NULL);
```



插入子查询结果 1

- 语句格式

INSERT

INTO <表名> [(<属性列1> [,<属性列2>...])

子查询;

- INTO子句

- 子查询

- SELECT子句目标列必须与INTO子句匹配

- ◆ 值的个数

- ◆ 值的类型



插入子查询结果 2

- [例3.72] 对每一个系，求学生的平均年龄，并把结果存入数据库
- 第一步：建表

```
CREATE TABLE Dept_age
( Sdept CHAR(15)           /*系名*/
  Avg_age SMALLINT);      /*学生平均年龄*/
```

- 第二步：插入数据

```
INSERT
INTO Dept_age(Sdept,Avg_age)
  SELECT Sdept, AVG(Sage)
FROM Student
GROUP BY Sdept;
```





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch19 SQL数据更新-修改

Software Institute
Nanjing University
Bei Jia

修改数据 1

- 语句格式

UPDATE <表名>

SET <列名>=<表达式>[,<列名>=<表达式>]...

[WHERE <条件>];

- 功能

- 修改指定表中满足WHERE子句条件的元组
- SET子句给出<表达式>的值用于取代相应的属性列
- 如果省略WHERE子句，表示要修改表中的所有元组



修改某一个元组的值

- [例3.73] 将学生201215121的年龄改为22岁

```
UPDATE Student
```

```
SET Sage=22
```

```
WHERE Sno=' 201215121 ';
```



修改多个元组的值

- [例3.74] 将所有学生的年龄增加1岁。

UPDATE Student

SET Sage= Sage+1;



带子查询的修改语句

- [例3.75] 将计算机科学系全体学生的成绩置零。

```
UPDATE SC
SET    Grade=0
WHERE Sno IN
      (SELETE Sno
      FROM    Student
      WHERE   Sdept= 'CS' );
```



修改数据与完整性

- 关系数据库管理系统在执行修改语句时会检查修改操作是否破坏表上已定义的完整性规则
 - 实体完整性
 - 参照完整性
 - 用户定义的完整性
 - NOT NULL约束
 - UNIQUE约束
 - 值域约束





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch20 SQL数据更新-删除

Software Institute
Nanjing University
Bei Jia

删除数据

- 语句格式

DELETE

FROM <表名>

[WHERE <条件>];

- 功能

- 删除指定表中满足WHERE子句条件的元组

- WHERE子句

- 指定要删除的元组
- 缺省表示要删除表中的全部元组，表的定义仍在字典中



删除某一个元组的值

- [例3.76] 删除学号为201215128的学生记录。

DELETE

FROM Student

WHERE Sno= 201215128 ';



删除多个元组的值

- [例3.77] 删除所有的学生选课记录。

DELETE

FROM SC;



带子查询的删除语句

- [例3.78] 删除计算机科学系所有学生的选课记录。

DELETE

FROM SC

WHERE Sno IN

(SELETE Sno

FROM Student

WHERE Sdept= 'CS') ;





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch21 SQL中的空值

Software Institute
Nanjing University
Bei Jia

空值的处理

- 空值就是“不知道”或“不存在”或“无意义”的值。
- 一般有以下几种情况：
 - 该属性应该有一个值，但目前不知道它的具体值
 - 该属性不应该有值
 - 由于某种原因不便于填写



空值的产生 1

- 空值是一个很特殊的值，含有不确定性。对关系运算带来特殊的问题，需要做特殊的处理。
- 空值的产生
- [例 3.79]向SC表中插入一个元组，学生号是"201215126"，课程号是"1"，成绩为空。

```
INSERT INTO SC(Sno,Cno,Grade)
```

```
VALUES('201215126 ','1',NULL); /*该学生还没有考试成绩，取空值*/
```

- 或

```
INSERT INTO SC(Sno,Cno)
```

```
VALUES(' 201215126 ','1'); /*没有赋值的属性，其值为空值*
```



空值的产生 2

- [例3.80] 将Student表中学生号为“201215200”的学生所属的系改为空值。

UPDATE Student

SET Sdept = NULL

WHERE Sno='201215200';



空值的判断

- 判断一个属性的值是否为空值，用IS NULL或IS NOT NULL来表示。
- [例 3.81] 从Student表中找出漏填了数据的学生信息

```
SELECT *
```

```
FROM Student
```

```
WHERE Sname IS NULL OR Ssex IS NULL OR  
Sage IS NULL OR Sdept IS NULL;
```



空值的约束条件

- 属性定义（或者域定义）中
 - 有NOT NULL约束条件的不能取空值
 - 加了UNIQUE限制的属性不能取空值
 - 码属性不能取空值



空值的算术运算、比较运算和逻辑运算 1

- 空值与另一个值（包括另一个空值）的算术运算的结果为空值
- 空值与另一个值（包括另一个空值）的比较运算的结果为UNKNOWN。
- 有UNKNOWN后，传统二值（TRUE，FALSE）逻辑就扩展成了三值逻辑



空值的算术运算、比较运算和逻辑运算 2

表3.8 逻辑运算符真值表

x	y	x AND y	x OR y	NOT x
T	T	T	T	F
T	U	U	T	F
T	F	F	T	F
U	T	U	T	U
U	U	U	U	U
U	F	F	U	U
F	T	F	T	T
F	U	F	U	T
F	F	F	F	T

T表示TRUE，F表示FALSE，U表示UNKNOWN



空值的算术运算、比较运算和逻辑运算 2

- [例3.82] 找出选修1号课程的不及格的学生。

```
SELECT Sno
```

```
FROM SC
```

```
WHERE Grade < 60 AND Cno='1';
```

- 查询结果不包括缺考的学生，因为他们的Grade值为null。



空值的算术运算、比较运算和逻辑运算 4

- [例 3.83] 选出选修1号课程的不及格的学生以及缺考的学生。

```
SELECT Sno
```

```
FROM SC
```

```
WHERE Grade < 60 AND Cno='1'
```

```
UNION
```

```
SELECT Sno
```

```
FROM SC
```

```
WHERE Grade IS NULL AND Cno='1'
```

- 或者

```
SELECT Sno
```

```
FROM SC
```

```
WHERE Cno='1' AND (Grade<60 OR Grade IS NULL);
```